

new/delete vs malloc/free

C++ հարցազրույցի ամբողջական ուղեցույց

Այս ուղեցույցը մանրամասն համեմատում է `new / delete`-ը (C++) և `malloc / free`-ը (C)՝ constructor/destructor, type safety, sizeof, exception, օրինակներով և հարցազրույցի հարցերով:

1. Հիմնական տարբերություններ

```
// C++ (new/delete)
int* p = new int(10);
delete p;

// C (malloc/free)
int* q = (int*)malloc(sizeof(int));
*q = 10;
free(q);
```

Ամենամեծ տարբերությունը՝ **new/delete-ը կանչում է constructor/destructor**, malloc/free-ն՝ ոչ:

2. Constructor / Destructor

new -> constructor կանչվում

```
class A {  
public:  
    A() { cout << "Constructor\n"; }  
    ~A() { cout << "Destructor\n"; }  
};  
  
A* p = new A();           // Output: Constructor  
delete p;                 // Output: Destructor
```

malloc -> constructor Չի կանչվում

```
A* q = (A*)malloc(sizeof(A)); // ՈՉ Մի Constructor -- միայն raw memory  
free(q);                      // ՈՉ Մի Destructor
```

`malloc` -ը միայն raw բայթեր է վերցնում -> object չի «կառուցվում» (member-ները չեն initialize): Class-երի համար սա վտանգավոր է:

3. Type Safety

```
int* p = new int;           // վերադարձնում է int* (type-safe)  
int* q = (int*)malloc(4);   // վերադարձնում է void* -> պետք է cast
```

`new` -ը վերադարձնում է ճիշտ type-ը (`int*`): `malloc` -ը վերադարձնում է `void*`, պետք է ձեռքով cast անել -> ավելի սխալապատ:

4. Size-ի հաշվարկ

```
int* p = new int[10];       // compiler-ն ինքն է գիտի չափը  
int* q = (int*)malloc(10 * sizeof(int)); // ձեռքով sizeof
```

`new` -ի համար չափը ավտոմատ է; `malloc` -ի համար ձեռքով `sizeof` -> հեշտ է սխալվել:

5. Exception vs nullptr

```
int* p = new int[1000000000]; // ԱՆՀԱՋՈՂՈՒԹՅԱՆ դեպքում -> throw std::bad_alloc
int* q = (int*)malloc(...);    // ԱՆՀԱՋՈՂՈՒԹՅԱՆ դեպքում -> վերադարձնում nullptr
```

`new` -ը անհաջողության դեպքում **throw** է անում (`std::bad_alloc`): `malloc` -ը վերադարձնում է `nullptr` (պետք է ձեռքով ստուգել):

6. Resize

```
// malloc-ը ունի realloc
int* q = (int*)malloc(10 * sizeof(int));
q = (int*)realloc(q, 20 * sizeof(int)); // չափը փոխել

// new-ը ՉՈՒՆԻ realloc -- պետք է նոր new + copy + delete
```

`malloc` -ը ունի `realloc` (չափը փոխել); `new` -ի համար չկա՝ պետք է նոր block, copy, հին delete (կամ օգտագործիր vector):

7. Համեմատական աղյուսակ

	new/delete	malloc/free
Լեզու	C++	C
Constructor/Destructor	ԱՅՈ	ՈՉ
Return type	ճիշտ type (int*)	void* (cast պետք)
Size-ի հաշվարկ	ավտոմատ	ձեռքով (sizeof)
Անհաջողություն	throw bad_alloc	return nullptr
Resize	չկա (նոր+copy)	realloc
Operator/function	operator	function
Overridable	այո (operator new)	ոչ

8. Կարևոր կանոններ

- new -> delete | new[] -> delete[]
- malloc -> free
- Մի եզրույթ: new-ը free-ով, կամ malloc-ը delete-ով -> undefined behavior

```
int* p = new int;  
free(p);           // Մեղադրանք -- undefined behavior  
  
int* q = (int*)malloc(4);  
delete q;          // Մեղադրանք -- undefined behavior
```

9. Երբ որն օգտագործել

- C++ կոդ -> new/delete (կամ ավելի լավ՝ smart pointers)
- C կոդ / C API -> malloc/free
- Modern C++ -> ոչ մեկը ուղիղ -> make_unique / make_shared / vector

10. Հարցազրույցի հարցեր և պատասխաններ

new vs malloc հիմնական տարբերությունը:

new-ը կանչում է constructor (և delete-ը՝ destructor); malloc/free-ն՝ ոչ. new-ը type-safe, malloc-ը void return:*

new անհաջողության դեպքում ինչ է անում:

throw std::bad_alloc (malloc-ը nullptr վերադարձնում):

Կարելի է new-ի հետ free օգտագործել:

Ոչ -> undefined behavior. new->delete, malloc->free:

Ինչո՞ւ new-ն ավելի լավ է class-երի համար:

Constructor-ը կանչում -> object ճիշտ initialize; malloc-ը միայն raw memory (member-ներ uninitialized):

delete vs delete[]:

new->delete, new[]->delete[]. Սխալ գույգը -> UB:

Modern C++-ում ինչպես:

Ոչ մի raw new/delete/malloc -> make_unique/make_shared/vector (RAII):

Ամփոփում (Cheat Sheet)

new/delete (C++):	malloc/free (C):
ctor/dtor կանչում	ՈՉ ctor/dtor
type-safe (int*)	void* (cast պետք)
size ավտոմատ	sizeof ձեռքով
throw bad_alloc	return nullptr
չկա realloc	նկի realloc
operator (overridable)	function

ԿԱՆՈՆ:

new -> delete | new[] -> delete[] | malloc -> free

Մի հԱՊՈՆԻՐ (new+free / malloc+delete -> UB)

MODERN C++: ոչ մեկը -> make_unique / make_shared / vector